OpenZeppelin | security

Stellar Contracts RC v0.7.0 Audit

Stellar

April 6, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 4

Scope 5

System Overview 7

Smart Accounts 7

Governance 7

Fixed-Point Math 8

Upgradeable 8

Access Control and Role Transfer 9

ZkEmail DKIM Registry 9

Security Model and Trust Assumptions 9

High Severity 11

H-01 Unbound context_rule_ids Allow Rule Selection Downgrade After Signature Collection 11

Medium Severity 11

M-01 Default FungibleBurnable Burns Bypass FungibleVotes Vote Accounting 11

M-02 Capped Extension Cap Check Can Be Bypassed When Minting Does Not Update Local Total Supply 12

M-03 Governor Missing queue Lifecycle Function for Proposal State Transition 13

M-04 Dynamic Quorum Override Silently Ignored in Proposal Lifecycle 14

M-05 Raw Signers' Keys Stored in Memory 15

M-06 Arbitrary External Calls Possible in do_check_auth 16

Low Severity 17

L-01 Expired Pending Transfers Can Block Renouncing Ownership and Admin Roles 17

L-02 Checked I256 Truncating Division Can Trap on MIN / -1 Overflow 17

L-03 transfer_from Emits a Non-SEP-41 Transfer Event Payload 18

L-04 Spending-Limit Policy Allows Default Rule 18

L-05 No On-Chain Mapping Between Signers and Policies and Their IDs 19

L-06 Spending Limit Policy Bypass By Specifying Negative Amount 19

L-07 No Proposer Description Binding in Governor::propose 20

L-08 Queueing Requirement Not Cached in Proposal Data 21

L-09 Potential Optimization in Signer Duplication Detection 22

Notes & Additional Information 23

N-01 Documentation Improvement Suggestions 23

N-02 Misleading Documentation 24

N-03 Naming Suggestion 26

N-04 canonicalize_key Requirements Not Entirely Followed 26
N-05 Lack of batch_canonicalize_key Implementations 26

Conclusion 28

Appendix 29
Issue Classification 29

Summary

Type	Library	Total Issues	21 (18 resolved)
Timeline	From 2026-03-02 To 2026-03-19	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	1 (1 resolved)
		Medium Severity Issues	6 (5 resolved)
		Low Severity Issues	9 (7 resolved)
		Notes & Additional Information	5 (5 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin conducted a differential audit of the OpenZeppelin/stellar-contracts repository with HEAD commit 239a2a7 against BASE commit 82f9711.

In scope were the following files:

```
packages/
├── access/src/
│   ├── access_control/
│   │   ├── mod.rs
│   │   └── storage.rs
│   ├── ownable/
│   │   ├── mod.rs
│   │   └── storage.rs
│   └── role_transfer/
│       ├── mod.rs
│       └── storage.rs
├── accounts/src/
│   ├── policies/
│   │   ├── mod.rs
│   │   ├── simple_threshold.rs
│   │   ├── spending_limit.rs
│   │   └── weighted_threshold.rs
│   ├── smart_account/
│   │   ├── mod.rs
│   │   └── storage.rs
│   └── verifiers/
│       ├── ed25519.rs
│       ├── mod.rs
│       └── webauthn.rs
├── contract-utils/src/
│   ├── math/
│   │   ├── i128_fixed_point.rs
│   │   ├── i256_fixed_point.rs
│   │   ├── mod.rs
│   │   └── wad.rs
│   ├── pausable/
│   │   └── mod.rs
│   └── upgradeable/
│       ├── mod.rs
│       └── storage.rs
├── governance/src/
│   ├── governor/
│   │   ├── mod.rs
│   │   └── storage.rs
│   ├── lib.rs
│   ├── timelock/
│   │   ├── mod.rs
```

```
│   │   └── storage.rs
│   └── votes/
│       ├── mod.rs
│       └── storage.rs
├── macros/src/
│   ├── access_control.rs
│   └── lib.rs
├── tokens/src/
│   ├── fungible/
│   │   ├── extensions/
│   │   │   ├── capped/
│   │   │   │   ├── mod.rs
│   │   │   │   └── storage.rs
│   │   │   ├── mod.rs
│   │   │   └── votes/
│   │   │       ├── mod.rs
│   │   │       └── storage.rs
│   │   ├── mod.rs
│   │   └── storage.rs
│   ├── non_fungible/
│   │   ├── extensions/
│   │   │   ├── enumerable/
│   │   │   │   └── storage.rs
│   │   │   ├── mod.rs
│   │   │   └── votes/
│   │   │       ├── mod.rs
│   │   │       └── storage.rs
│   │   ├── mod.rs
│   │   └── storage.rs
│   ├── rwa/
│   │   ├── claim_issuer/
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── identity_registry_storage/
│   │   │   └── storage.rs
│   │   └── storage.rs
│   └── vault/
│       └── storage.rs
└── zk-email/src/
    ├── dkim_registry/
    │   ├── mod.rs
    │   └── storage.rs
    └── lib.rs
```

***Update**: The fixes for the findings highlighted in this report have all been merged at commit [d55dd37](d55dd37).*

System Overview

The OpenZeppelin Stellar Contracts library is a modular, trait-based smart contract framework for the Stellar/Soroban blockchain. It provides reusable components for fungible and non-fungible tokens, access control, governance, and account abstraction. Contracts are composed by implementing library traits and selecting extension types at compile time, giving integrators fine-grained control over which features they include while enforcing mutual exclusivity constraints through Rust's type system.

Smart Accounts

The smart accounts module implements a context-centric authorization framework that separates authentication into three independent dimensions: signers (who is signing), context rules (what operations are being authorized), and policies (how authorization is validated). When a smart account receives an authorization request through Soroban's `__check_auth` entry point, it authenticates all provided signers, matches each invocation context to its corresponding context rule (selected explicitly via `AuthPayload::context_rule_ids`), and then delegates validation to any policies attached to those rules.

This diff introduces several structural changes to the module. The `can_enforce` pre-check and `get_context_rules` auto-discovery methods were removed from the `Policy` trait, collapsing the two-phase validation model into a single `enforce` call. Callers must now supply explicit rule IDs for every context in the authorization payload. The module also adds `batch_add_signer` for atomic multi-signer registration, key canonicalization to detect duplicate external signers across different encodings, signer key size limits (256 bytes), and deterministic fingerprinting of context rules for deduplication. A global signer and policy registry with reference counting supports efficient storage management.

Governance

The governance module provides a full on-chain governance system. On top of the existing Timelock controller submodule, two additional submodules have been introduced: the Governor module for proposal lifecycle management and the Votes module for checkpoint-based historical voting power tracking.

The Governor manages proposals through a state machine with time-derived transitions. Proposals are created with a deterministic identifier (keccak256 hash of targets, functions, arguments, and description hash) and record a snapshot ledger at creation time, equal to the current ledger increased by the voting delay period. Voting always starts after this snapshot and all voting power lookups reference this snapshot, preventing flash-loan-based voting manipulation. The counting mechanism supports three vote types (Against, For, Abstain), and a proposal succeeds when participation (For plus Abstain votes) meets the quorum and For votes strictly exceed Against votes. The library does not provide default implementations for the `execute` and `cancel` actions, requiring integrators to define their own access control for these operations.

The Votes module tracks voting power through a checkpoint timeline per delegate, supporting historical lookups via binary search. Voting power is delegation-based: holding tokens alone does not grant voting power, and accounts must explicitly delegate (including self-delegation) for their units to count. Token Votes extensions for both fungible and non-fungible tokens hook into transfer, mint, and burn operations to keep voting unit balances synchronized.

Fixed-Point Math

The fixed-point math module was refactored from a trait-based design to standalone free functions (`mul_div_floor`, `mul_div_ceil`, `mul_div`, and their `checked_` variants) for both `i128` and `I256`. The `i128` path handles phantom overflow by transparently retrying the computation using `I256` as an intermediate type. The `I256` variant does not apply phantom overflow handling (no `I512` type is available), which is documented as an accepted trade-off. Division by zero results in panics via the standard library instead of returning a typed `DivisionByZero` error.

Upgradeable

The upgradeable module was rearchitected to a minimal interface. The `Migratable` trait and all its associated derive macros were removed. In their place, a simple `Upgradeable` trait provides only the `upgrade` entry point, paired with `get_schema_version` and `set_schema_version` helpers for migration guards. Migration is now documentation-driven, with three recommended patterns: eager migration for bounded data, lazy migration for unbounded data, and enum wrappers for forward-compatible storage layouts.

Access Control and Role Transfer

The access control module received a namespace cleanup (functions now use a `storage::` prefix) with no logic changes. The role transfer mechanism replaces bare `Address` storage with a `PendingTransfer` struct that records both the recipient and an explicit `live_until_ledger` deadline. This ensures the transfer deadline is enforced independently of storage TTL, preventing scenarios where permissionless TTL extensions could keep an expired transfer alive. A new `TransferExpired` error is returned when accepting a transfer past its deadline.

ZkEmail DKIM Registry

The DKIM registry is a new module implementing an on-chain trust anchor for zero-knowledge email proofs, inspired by the ERC-7969 pattern. It stores mappings of domain hashes to public key hashes, supporting registration, batch registration, and revocation. The module is hash-function agnostic: callers hash values off-chain before submitting them. The base implementation intentionally omits authorization checks on all mutating operations, leaving access control to the integrator.

Security Model and Trust Assumptions

The library follows a dual-layer API pattern where high-level functions include authorization checks, event emission, and validation, while low-level functions provide granular control without automatic guards. Security-sensitive decorators (`#[only_owner]`, `#[when_not_paused]`, `#[only_role]`) are applied as procedural macro attributes. Several modules, including the DKIM registry and the Governor's `execute`/`cancel` entry points, deliberately defer access control to the integrator.

The following trust assumptions are critical to the security of the system:

- Context rules are expected to be correctly configured in contracts relying on the smart accounts library. In particular, every rule targeting a policy contract will have stricter execution requirements than the ones defined by that policy. Otherwise, it would enable

privilege escalation attacks where requirements for a policy, and thus for any context rule relying on that policy, could be downgraded by using weaker context rules.

- Weighted-threshold policy administrators are expected to only configure `signer_weights` with well-formed signers that respect the smart-account external key-size expectations and will not intentionally add oversized or irrelevant external keys. Otherwise, policy state may be bloated in ways that increase authorization cost and may make enforcement/read paths fail under resource limits.

- `signer_weights` is expected to be kept synchronized with `ContextRule.signers` so threshold reachability checks reflect the signer set actually usable at runtime. Otherwise, stale or extraneous weights can make admin validation pass while `enforce` still fails deterministically for that rule.

- `ContextRuleType::CallContract` scopes rules to a contract address only, discarding the function name and arguments present in Soroban's `Context::Contract`. A rule that authorizes calls to a given contract therefore authorizes all `require_auth`-gated entrypoints on that contract (including admin mutations), not just the intended function. Threshold policies that do not inspect `Context` (e.g. `simple_threshold`, `weighted_threshold`) cannot compensate for this, so integrators must treat `CallContract` rules as granting full contract-level access and rely on per-function authorization logic in the target contract itself.

- External signer verifier contracts are assumed to behave correctly. Malicious or buggy verifiers called during the authentication phase of `__check_auth` could block authorization entirely or produce incorrect verification results.

- The DKIM registry's mutating operations (`set_dkim_public_key_hash`, `set_dkim_public_key_hashes`, `revoke_dkim_public_key_hash`) carry no built-in authorization. Integrators are responsible for wrapping these with appropriate access control. Failure to do so may allow any caller to register or revoke DKIM key hashes.

- Governor integrators must provide their own access control for `execute` and `cancel`, as these functions have no default implementation. Open execution models allow any caller to trigger approved proposals, which is a deliberate design choice but must be an informed one.

- Instance storage TTL extension is the responsibility of the contract developer. The library manages TTL for its own persistent storage entries, but the account contract's instance storage can expire if not extended by the integrator.

High Severity

H-01 Unbound `context_rule_ids` Allow Rule Selection Downgrade After Signature Collection

The `do_check_auth` function authenticates signatures over `signature_payload` and then validates each `auth_context` against a caller-supplied rule ID (`AuthPayload.context_rule_ids`). The host-computed `signature_payload` includes `network_id`, `nonce`, `expiration_ledger`, and the authorized invocation tree (contract, fn, args).

However, it does not include the custom `AuthPayload` or any rule IDs. As a result, a sponsor can alter `context_rule_ids` after signatures are collected, selecting a weaker rule (fewer signers or fewer policies) for the same invocation. Because `get_validated_context_by_id` trusts the supplied rule ID and `do_check_auth` only enforces policies for the selected rule, authorization succeeds under the downgraded rule without invalidating signatures. This defeats protections like spending limits or threshold policies that signers expected to apply.

Consider binding the selected rule IDs to what signers authorize (e.g., include a hash of context_rule_ids in the signed payload or `require_auth_for_args` on a payload that includes rule IDs).

Update: *Resolved in pull request #655.*

Medium Severity

M-01 Default `FungibleBurnable` Burns Bypass `FungibleVotes` Vote Accounting

The governance votes module uses a checkpoint-based model where `get_votes` returns the latest stored checkpoint rather than deriving voting power from token balances at read time. Checkpoints are only updated when the token implementation calls

`transfer_voting_units`. The module documents that integrators must update voting units on every balance change, including mints, burns, and transfers.

`FungibleVotes` correctly hooks voting-unit updates for transfers by implementing `ContractOverrides` and calling `transfer_voting_units` after the base transfer. It also provides vote-aware `burn` and `burn_from` helpers that call `transfer_voting_units` after the base burn. However, the burn entrypoints exposed by `FungibleBurnable` do not use these helpers. The default `FungibleBurnable::burn` and `burn_from` call `Base::burn` and `Base::burn_from` directly, which update balances and total supply but never call `transfer_voting_units`. Hence, composing `FungibleVotes` with default `FungibleBurnable` creates a footgun for developers where `burn` and `burn_from` will break vote accounting.

Consider adding a separate `BurnableOverrides` trait for fungible tokens, following the pattern already used for NFT burnable overrides, rather than extending the existing `ContractOverrides`. `FungibleBurnable` should then require `ContractType: BurnableOverrides` and delegate `burn`/`burn_from` to the contract type implementation, allowing `FungibleVotes` (and future extensions) to hook into burn flows.

Update: *Resolved in pull request #622.*

M-02 Capped Extension Cap Check Can Be Bypassed When Minting Does Not Update Local Total Supply

The capped extension's `check_cap` enforces the supply cap by reading `FungibleStorageKey::TotalSupply` directly from instance storage and comparing the sum of the current supply and the requested mint amount against the configured cap. When minting is delegated to an external backend (for example via `sac_admin_wrapper::mint`, which forwards to `StellarAssetClient::mint`), the local `TotalSupply` key is never updated. In this configuration, `check_cap` reads a stale value (defaulting to 0 if unset), allowing repeated mints to cumulatively exceed the cap without reverting. Overriding `FungibleToken::total_supply` to return an authoritative backend value does not help, since `check_cap` reads the raw storage key rather than calling the trait method.

Consider introducing a `check_cap` variant that accepts the current total supply as an explicit argument or calls an overridable supply hook, so that integrators using external mint backends can provide the authoritative value. At minimum, consider documenting that `check_cap` is

only valid when the local `TotalSupply` key is kept synchronized by the mint path (i.e., when using `Base::update` rather than an external backend).

Update: *Resolved in [pull request #662](#).*

M-03 Governor Missing `queue` Lifecycle Function for Proposal State Transition

The Governor module defines `Queued` and `Expired` as explicit `ProposalState` variants and handles them consistently at every internal branch point. The state derivation function preserves both states, `execute` gates on `Queued` when the queue flag is enabled, `cancel` permits cancellation of queued proposals, and a dedicated `emit_proposal_queued` event helper is available.

However, no corresponding `queue` storage function exists to produce the `Queued` state. Every other lifecycle transition has a dedicated function (`propose` sets `Pending`, `execute` sets `Executed`, `cancel` sets `Canceled`), but no function performs the `Succeeded` to `Queued` transition.

This gap has several consequences. An extension author who overrides `proposal_needs_queuing` to return `true` must manually read `ProposalCore` through internal storage keys, set the state field to `Queued`, and write it back, with no transition validation enforcing that the proposal is actually in the `Succeeded` state first. The module-level documentation states that overriding `proposal_needs_queuing` to return `true` is sufficient to wire up the full queuing flow, but this only configures the execution gate without providing or demonstrating the mechanism to reach the `Queued` state. The Timelock module further compounds this by operating an entirely separate state `OperationState` that has no bridge to `ProposalState`. The existing timelock-controller example contains no references to queuing, `ProposalState`, or `proposal_needs_queuing`. As a result, enabling queuing without a complete custom extension can be error-prone: proposals that pass voting reach derived `Succeeded` status, but `execute` panics with `ProposalNotQueued` because nothing transitions the proposal to `Queued`, and the only recovery path is cancellation.

Consider implementing a `queue` storage function that validates the `Succeeded` precondition before writing the `Queued` state, matching the pattern established by `execute` and `cancel`. This would prevent extensions from transitioning proposals out of invalid states and remove the need for extension authors to manipulate internal storage layout directly. If the intent is to leave the transition entirely to implementors, consider updating the module

documentation to explicitly describe the required storage manipulation steps and providing an example that wires the Governor and Timelock modules together end-to-end.

Update: *Resolved in pull request #659 and pull request #671.*

M-04 Dynamic Quorum Override Silently Ignored in Proposal Lifecycle

The `Governor` trait exposes a `quorum` method documented as an extension point for dynamic quorum strategies.

However, when a proposal is created, `storage::propose` snapshots the quorum by calling `get_quorum`, which reads the raw value from instance storage. This value is stored in `ProposalCore.quorum` and later used by `derive_proposal_state` and `quorum_reached` to determine whether a proposal succeeded. Since `storage::` functions are free functions, they cannot dispatch through `Self::quorum` and instead always use the storage value. An implementer who overrides the trait method to return a supply-relative quorum while leaving a low placeholder in storage would observe the following:

1. The overridden `Governor::quorum` returns a dynamic value (e.g., 4% of total supply).
2. External callers and governance UIs query the trait method and display the dynamic quorum.
3. Internally, `storage::propose` snapshots the placeholder value from instance storage into the proposal.
4. Proposals pass with participation far below the intended threshold, while the UI shows the higher requirement.

The failure mode is silent: no error is raised when the override is ineffective, and the discrepancy between the displayed and enforced quorum is not surfaced to governance participants.

Consider having `storage::propose` accept an explicit `quorum` parameter instead of reading from instance storage. The default `Governor::propose` implementation can then pass `Self::quorum(e, vote_start)`, allowing dynamic quorum overrides to propagate into the proposal lifecycle.

Update: *Resolved in pull request #647 and pull request #670. The governor quorum flow was reworked so quorum is no longer snapshotted as a raw per-proposal storage value at proposal creation. Instead, quorum is derived for each proposal at its `vote_snapshot` and threaded through proposal lifecycle operations, while static quorum values are now checkpointed over*

ledger time so later quorum updates do not retroactively affect earlier proposals. This also allows dynamic quorum overrides to participate in proposal evaluation at the correct snapshot, provided custom implementations handle future `vote_snapshot` *lookups safely.*

M-05 Raw Signers' Keys Stored in Memory

During signer registration, `contains_canonical_duplicate` canonicalizes external keys (via `VerifierClient.batch_canonicalize_key`) to prevent duplicate cryptographic identities even when raw bytes differ. However, signers' keys themselves are stored as raw bytes, which causes several issues.

First, during authorization, `get_authenticated_signers` matches signers by raw equality only. This means a signer can be valid and verifiable, yet fail to match the stored rule signer if the key is presented in a different but canonical-equivalent encoding. This can happen for external signers whose key data allows multiple representations (e.g., WebAuthn key bytes with optional credential ID suffix). The signature can verify, but the signer is not counted as authenticated, so policy enforcement or rule validation fails.

Second, the `verify` function of each verifier is called on raw key_data, so for example if a WebAuthn key with optional `credential_id` suffix has been registered, `webauthn::verify` will fail as it expects to receive 65-byte keys.

Finally, the `register_signer` function computes fingerprints based on raw bytes. Hence, two signers sharing the same verifier and the same canonical key, but differing in representations will have different `SignerEntry` structs. As a result, two identical rules may have different fingerprints, which may result in redundant storage entries created.

Consider storing canonicalized key bytes at registration and canonicalizing signer keys during authentication before matching against rule signers.

Update: *Acknowledged, not resolved. The OpenZeppelin development team stated:*

> *Storing canonical keys seems logical, but deciding not to is a deliberate design choice.*
>
> *It's mostly related to the practical implications when dealing with passkeys. When a client generates a passkey, it returns a r1 key + a credential ID. The r1 key is only available during this registration step and one can no longer access it afterwards. The smart account needs the r1 key only to check the signature. That forces implementors to store off-chain a mapping (r1 key, ID), then to use the ID when prompting the user to sign a payload while using the r1 key for identifying the smart account's* `Signer`*. We can argue this can be very well handled off-chain, but we deem necessary to store the*

> *whole on-chain, because if one loses their credential ID the r1 key becomes almost unusable.*
>
> *We're deciding to stick with the current design, i.e. using canonical representations only for the purpose of duplicates verification when keys get added. Regarding the concerns listed in the report:*
>
> *- `The signature can verify, but the signer is not counted as authenticated, so policy enforcement or rule validation fails.`: It's up to the client to submit a key format expected by the smart account, this key is stored on-chain so the client has an easy way to access it.*
>
> *- `if a WebAuthn key with optional credential_id suffix has been registered, webauthn::verify will fail as it expects to receive 65-byte keys.`: `webauthn::verify` is a storage function and the example verifier demonstrates how to use it.*
>
> *- `two identical rules may have different fingerprints, which may result in redundant storage entries created`: Fingerprints were removed with this PR.*

M-06 Arbitrary External Calls Possible in `do_check_auth`

The `do_check_auth` function is the core smart-account authorization path invoked by `__check_auth`: it authenticates signatures, validates selected rules against each auth context, and then enforces policies. During authentication, every external signer triggers an external verifier call via the `VerifierClient::verify(...)` call.

However, `do_check_auth` authenticates all signers in `AuthPayload.signers` before filtering to signers that are actually relevant to the selected rules and the rule-based signer filtering only happens later via `get_authenticated_signers`. This allows an attacker to append non-authorizing external signers with attacker-controlled verifiers, causing side effects before the protected flow continues (e.g., DEX price manipulation during auth), even though those extra signers do not count toward final authorization.

Consider deriving the allowed signer set from selected `context_rule_ids` first, and reverting if `AuthPayload.signers` contains any signer outside that set, so only rule-relevant signers are authenticated. In practice, this means moving signer relevance checks

ahead of external verifier calls to remove unnecessary untrusted external execution from the auth path.

Update: *Resolved in pull request #656 and pull request #667.*

Low Severity

L-01 Expired Pending Transfers Can Block Renouncing Ownership and Admin Roles

The `renounce_ownership` and `renounce_admin` functions prevent renouncing while a pending transfer exists by checking whether the pending storage key holds a value. This guard does not inspect the `live_until_ledger` field stored inside the `PendingTransfer` entry, so an expired pending transfer blocks renouncing just as an active one would. When the current ledger exceeds `live_until_ledger`, `accept_transfer` reverts with `TransferExpired` before removing the entry. Because Soroban temporary storage entries can remain live past the explicit deadline due to permissionless TTL extension, the expired entry persists in storage. The owner can still work around this by calling `transfer_role` with `live_until_ledger = 0` to explicitly cancel the stale entry, but this workaround is not documented and the revert message (`TransferInProgress`) is misleading for an expired transfer.

Consider updating the renounce guards to also check `live_until_ledger`, treating expired entries as absent and removing them as cleanup.

Update: *Resolved in pull request #648.*

L-02 Checked `I256` Truncating Division Can Trap on MIN / -1 Overflow

The `checked_mul_div` function is the non-panicking variant of `mul_div` for `I256` fixed-point arithmetic. It guards against division by zero by returning `None` when `denominator` is zero, but it does not check for the `I256::MIN / -1` division overflow before calling `r.div(denominator)`. The floor and ceil checked variants (`checked_div_floor` and `checked_div_ceil`) both use the `check_div_overflow` helper for this case, but the truncating path reached via `Rounding::Truncate` in

`checked_mul_div_with_rounding` skips it. When `x * y` evaluates to `I256::MIN` and `denominator` is `-1`, the result cannot be represented in a signed 256-bit integer and `I256::div` traps. This violates the documented checked-API contract that promises `None` on division overflow.

Consider adding the same `check_div_overflow` guard to `checked_mul_div` before the division, consistent with the floor and ceil paths.

Update: *Resolved in pull request #645.*

L-03 `transfer_from` Emits a Non-SEP-41 Transfer Event Payload

The fungible token module emits all transfer events through a shared `emit_transfer` helper, which publishes the `Transfer` struct containing both `to_muxed_id: Option<u64>` and `amount: i128` as event data fields. `Base::transfer_from` calls this helper with `to_muxed_id` set to `None`, producing a two-field struct payload.

However, the documentation of this function specifies event data as `[amount: i128]` and the SEP-41 standard requires a single `i128` in the `transfer_from` event data. As a consequence, off-chain consumers that decode the data field as a bare `i128` will fail to parse the struct-encoded payload.

Consider emitting a dedicated event payload with a single `i128` data field for `transfer_from` to match the documented schema and SEP-41 convention, or aligning the documentation to reflect the actual struct-based payload if the muxed schema is intentional for all transfer paths.

Update: *Resolved in pull request #646.*

L-04 Spending-Limit Policy Allows `Default` Rule

The spending-limit policy is intended to enforce a rolling-window quota by tracking usage in persistent per-`(account, rule)` state and rejecting over-limit calls. The remaining limit is decreased on each policy `enforce` call where the target function name equals `transfer` and that function has at least 3 arguments.

However, this policy is allowed to be used when applying the `Default` context rule, which can be used to call any contract, including arbitrary token contracts. This can result in a

situation where transferring two different tokens, possibly with different decimals number, can be counted towards the same spending limit.

Consider rejecting the `Default` context rules in the `install` function of the spending-limit policy and documenting that this context rule type is not suitable for this policy.

Update: *Resolved in pull request #649 at commit dae1381.*

L-05 No On-Chain Mapping Between Signers and Policies and Their IDs

Context rules are stored as a `ContextRuleEntry` that contains `signer_ids` and `policy_ids`, while the public `ContextRule` returned by `add_context_rule` and `get_context_rule` contains only resolved signers and policies. The `signer_ids`/`policy_ids` are only documented as part of the `add_context_rule` event schema.

However, no view method returns the stored ID vectors, and no method maps a `Signer` or policy `Address` to its corresponding ID within a rule. This prevents purely on-chain controllers from removing a specific compromised signer or policy unless the IDs were stored off-chain or during creation.

Consider exposing view methods that return the stored ID vectors (for example, a `get_context_rule_entry` returning `ContextRuleEntry`, or dedicated `get_signer_ids`/`get_policy_ids` functions). Consider also adding value-based on-chain lookup helpers that map `(context_rule_id, Signer)` and `(context_rule_id, Address)` to IDs.

Update: *Resolved in pull request #664.*

L-06 Spending Limit Policy Bypass By Specifying Negative Amount

The spending-limit policy's `enforce` function currently parses the transfer `amount` as `i128` and uses it directly in accounting without enforcing that `amount` is strictly positive.

However, while the fungible token implementation for this library does revert on attempts to transfer negative amounts of tokens, such behavior is not required by the SEP-41 standard and as such, another implementation could potentially treat such transfer as 0 token transfer. As a result, if a `transfer` call is accepted with a negative amount by the target token

implementation, the policy will reduce `cached_total_spent`, effectively increasing remaining spend capacity instead of consuming it.

Consider validating `amount >= 0` inside the policy itself to make enforcement independent of token-specific semantics.

Update: *Resolved in pull request #649 at commit 5a70000.*

L-07 No Proposer Description Binding in `Governor::propose`

The governor's `propose` function currently computes proposal IDs deterministically from `targets`, `functions`, `args`, and `description_hash` and accepts the proposal as long as that ID does not already exist.

However, proposer identity is not bound to the description or proposal ID derivation. The `Governor::propose` entrypoint authenticates the proposer, but no proposer-description restriction is validated before ID computation and uniqueness checks in storage. Therefore, if proposal payloads are observable before inclusion, another account can submit the same payload first and claim the proposal ID, which can affect proposer-attributed governance behavior and metadata assumptions.

Consider adding an optional proposer-description binding check, similar to the one from OpenZeppelin's Solidity `Governor` implementation. This optional check should be reflected in the documentation.

Update: *Acknowledged, not resolved. The OpenZeppelin development team stated:*

> *We don't agree with the finding, due to:*
>
> ***1. The proposer is already stored on-chain.*** *Once a proposal is successfully created, the `proposer` field in `ProposalCore` is set and used for cancellation rights and any proposer-gated logic. There is no privilege escalation or access control bypass — the recorded proposer always matches whoever actually submitted the transaction.*
>
> ***2. The attack outcome is functionally equivalent.*** *If Bob front-runs Alice's proposal with an identical payload, the governance action that gets voted on and executed is the same. The protocol behaves correctly regardless of who submitted it. Alice can simply resubmit with a slightly different description, producing a new proposal ID.*

> ***3. Stellar's transaction model reduces the likelihood significantly.*** *Unlike Ethereum's public mempool, Stellar does not expose pending transactions in a way that makes front-running a practical attack vector. This substantially reduces the real-world exploitability of this issue.*
>
> ***4. The mitigation introduces non-trivial complexity for marginal benefit.*** *Implementing the OZ-style proposer-description binding requires a new description format convention, additional validation logic in* `propose`*, and documentation explaining the opt-in mechanism. This adds surface area to an already complex governance module, for protection against a low-likelihood scenario with no impact on governance correctness or security.*
>
> ***5. There's also the proposer address in the event so this can be cross-checked on the client side if this is important.***
>
> ***Severity assessment:*** *Informational. No funds at risk, no access control bypass, no impact on governance outcomes. We consider this an acceptable trade-off*

L-08 Queueing Requirement Not Cached in Proposal Data

The governor's `execute` function currently derives queueing requirements from the `queue_enabled` runtime parameter and uses it to choose between `Queued` and `Succeeded` gating.

However, the `queue_enabled` parameter is expected to be based on the value returned by the `proposal_needs_queuing` function, which does not accept the proposal ID, hence it cannot return a proposal-specific value.

Consider changing the `proposal_needs_queuing` function's signature to allow making queueing decisions specific to a given proposal.

Update: *Acknowledged, not resolved. The OpenZeppelin development team stated:*

> *We've evaluated this finding and decided to keep* `proposal_needs_queuing` *as a global setting rather than introducing per-proposal queueing logic. Here's our reasoning:*
>
> ***1. The current design is intentional, not an oversight.*** *The* `proposal_needs_queuing` *trait method returns a global boolean by design. A governor instance either uses a timelock for all proposals or it doesn't. This is*

> *the simplest and most predictable model for governance participants — there's no ambiguity about whether a given proposal will be time-delayed.*
>
> ***2. Emergency execution is better handled through separate governance paths.*** *The primary use case for per-proposal queueing is emergency actions that need to bypass the timelock. This is more safely achieved by deploying a separate governor with different parameters (shorter voting period, higher quorum, no timelock) that is only authorized to call a narrow set of emergency functions (e.g., pause). The target contract's access control enforces the permission boundary, which is simpler and more auditable than proposal classification logic inside the governor.*
>
> *Our architecture already supports this pattern without any code changes — the timelock controller and governor are separate contracts, and multiple governors can coexist with different configurations.*
>
> ***3. Per-proposal queueing introduces a security-sensitive classification problem.*** *If* `proposal_needs_queuing` *inspects proposal content to decide whether queueing is required, the classification logic itself becomes security-critical. A misclassification — whether due to a bug or a cleverly crafted proposal — would allow a proposal to bypass the timelock entirely, undermining the protection it provides. Keeping queueing as a global invariant eliminates this attack surface.*
>
> ***Severity assessment:*** *Informational. The current design follows a deliberate architectural pattern where queueing is a property of the governor instance, not of individual proposals. Emergency governance scenarios are supported through deployment configuration rather than runtime classification.*

L-09 Potential Optimization in Signer Duplication Detection

The `add_context_rule` function validates duplicate signers by iterating through input signers and calling `contains_canonical_duplicate` against an incrementally built `unique_signers` vector. For external signers, this helper performs verifier-side canonicalization through `batch_canonicalize_key`, so duplicate detection involves external contract calls.

However, the current loop in `add_context_rule` may trigger repeated canonicalization calls for each signer insertion attempt, causing avoidable extra external calls and superlinear work as signer count grows. While bounded by `MAX_SIGNERS`, this still increases authorization-

management cost and exposes unnecessary verifier interaction surface during rule creation/update flows. Similar issue applies to the `batch_add_signer` function.

For each context rule, consider storing canonicalized signers keys instead of raw ones and whenever new external signers are added, consider canonicalizing their keys and perform duplicate checks on the already canonicalized keys list from the storage, without a necessity to perform multiple calls to the verifier contract.

Update: *Resolved in pull request #657 at commit c301154.*

Notes & Additional Information

N-01 Documentation Improvement Suggestions

Throughout the codebase, there are several places where the documentation could be improved: - In `policies::Policy`, the docs explicitly define authorization expectations for `enforce`, but `install`/`uninstall` (which also mutate policy state and receive `smart_account`) do not have corresponding authorization guidance. This leaves a security-critical lifecycle requirement implicit and easy for third-party implementers to miss. - In `governor::storage::cancel` and the module-level queueing flow docs (`governor::mod`), cancellation is described only at governor-state level, with no mention of coordinating queued timelock operations (e.g., `timelock::storage::cancel_operation`). Consider adding a note to clarify that the integrator's cancel implementation must also cancel the corresponding timelock operation. - In the governance votes module, delegation is represented as optional in reads (e.g., `get_delegate` returns `Option<Address>`), but the write path only exposes delegation to a concrete address (`delegate(..., delegatee: Address)`), so there is no explicit "delegate to `None`" / undelegate operation. This behavior should be documented clearly to avoid integrator confusion.

Consider addressing the points above in order to improve the clarity of the codebase.

Update: *Resolved in pull request #658 at commit 97f97d6, c8387a6 and at commit 5499e83.*

N-02 Misleading Documentation

Throughout the codebase, there are multiple instances of misleading or incomplete documentation. For example: - In `access_control::mod`, the doc blocks for `accept_admin_transfer` and `transfer_admin_role` appear swapped. - In `governor::storage::execute`, docs mention an `executor` argument, but the function takes `queue_enabled: bool`.

- In `governor::storage::cancel`, docs mention an `operator` argument, but the function signature has no such parameter.
- In `governor::storage::check_proposal_state`, docs mention a `voter` argument, but the signature only takes `proposal_id`.
- In `smart_account::emit_context_rule_added`, docs describe a single `context_rule` argument, but the function signature uses decomposed fields (`context_rule_id`, `name`, `context_type`, `valid_until`, `signer_ids`, `policy_ids`).
- In `smart_account::storage::authenticate`, docs refer to `signatures`, while the function parameter is named `signers`.
- In `i128_fixed_point`, several `# Arguments` sections omit the `x` parameter for functions whose signatures include it (e.g., `mul_div_floor`, `mul_div_ceil`, `mul_div`, and checked variants).
- In `non_fungible::storage::approve_for_owner`, the docs omit the `owner` argument even though it exists in the signature.
- In `non_fungible::storage::decrease_balance`, docs describe decreasing balance of `to`, but the function parameter is `from`.
- In `enumerable::storage::add_to_enumerations`, `# Arguments` documents only `e`, while signature also requires `owner` and `token_id`.
- In `math::mod`, docs state panicking variants panic with `SorobanFixedPointError`, but panicking implementations now rely on native arithmetic panics (e.g., `i128_fixed_point::mul_div`, `i256_fixed_point::mul_div`).
- In `i256_fixed_point::checked_mul_div_floor` and `checked_mul_div_ceil`, docs say these return `None` on invalid division, but both

compute `x.mul(y)` before denominator checks in helpers (`checked_div_floor`/`checked_div_ceil`), so multiplication overflow can panic before `None` is returned.

- In `i128_fixed_point` and `i256_fixed_point`, some intra-doc links point to renamed/non-existent symbols (e.g., `mul_div_floor_i128`, `mul_div_with_rounding_i256`), resulting in broken or misleading rustdoc references.
- In `smart_account::mod`, the documentation introduces "three key dimensions:" but does not enumerate them inline after the colon, which reduces clarity.
- In smart_account::storage::remove_context_rule, the comment states that `try_uninstall` "prevents a malicious or misconfigured policy from blocking context rule removal," but this is only partially true: it handles recoverable policy failures, while a malicious policy can still cause non-recoverable failures (e.g., storage limit exhaustion) that revert the transaction; the docs should frame this as a trust assumption and describe uninstall as best-effort.
- `#[has_role]` only checks role membership and does not authenticate the passed address. The example in `access_control mod.rs` can be unsafe if copy-pasted without `require_auth` (or `#[only_role]`) on that address.
- In `governor::mod`, documentation states that voting power is snapshotted at proposal creation, but implementation snapshots it at `vote_start` (`current_ledger + voting_delay`).
- In `governor::storage::hash_proposal` and `Governor::get_proposal_id`, docs omit that proposal hashing uses XDR-serialized inputs (in particular, `description_hash` is computed from `description.to_xdr(e)`). As a result, off-chain clients that hash plaintext description bytes (or otherwise skip matching XDR encoding) can compute incompatible proposal IDs and hit `ProposalNotFound`.

Consider fixing the instances of misleading documentation mentioned above in order to improve the clarity of the codebase.

Update: *Resolved in pull request #658 at commit b623f5a and at commit 0547e89.*

N-03 Naming Suggestion

The `vote_start` member of the `ProposalCore` struct is named as if it was the first active voting ledger, but in practice it is used as the snapshot ledger for voting power, where voting opens at `vote_start + 1`.

Consider renaming this variable to `snapshot` / `vote_snapshot` (and aligning related APIs/events) to better reflect its role.

Update: *Resolved in pull request #660.*

N-04 `canonicalize_key` Requirements Not Entirely Followed

The documentation of the `Verifier::canonicalize_key` function currently says that implementations should panic when `key_data` is malformed or when it does not represent a valid key.

However, the shared canonicalization helpers do not perform this required full cryptographic key validation: `ed25519::canonicalize_key` returns bytes from `BytesN<32>` as-is, and `webauthn::canonicalize_key` only extracts the first 65 bytes. This creates a mismatch between trait requirements and practical implementation behavior.

Consider relaxing the trait requirement to require panic only for malformed/invalid-length encodings, while defining identity guarantees over valid keys. Full key validity can remain enforced in `verify` (or optionally at registration via proof-of-possession), which avoids pushing expensive or hard-to-port cryptographic parsing requirements into canonicalization.

Update: *Resolved in pull request #657 at commit e22cbd3.*

N-05 Lack of batch_canonicalize_key Implementations

The `Verifier::batch_canonicalize_key` API explicitly requires preserving input order.

However, shared verifier helpers expose only single-key canonicalization (`ed25519::canonicalize_key` and `webauthn::canonicalize_key`), without a shared batched helper enforcing order semantics. This split increases maintenance risk, as future verifier implementations may still call `canonicalize_key` but accidentally reorder outputs

(for example via sorting), violating trait expectations and potentially breaking index-based duplicate detection flows that assume positional correspondence between input and canonicalized output.

Consider adding shared `batch_canonicalize_key` helpers in the verifier modules in order to lower the risk of not following the ordering requirement in the implementation of this function.

Update: *Resolved in pull request #657 at commit aca9e9a.*

Conclusion

This audit reviewed the OpenZeppelin Stellar Contracts RC v0.7.0 update, which expands a modular Soroban framework across smart accounts, governance, fixed-point math, upgradeability, access control/role transfer, and DKIM registry capabilities. The highest-severity findings identified during this review were concentrated in insufficient validation inside smart-account authentication checking logic, and current governance design limitations around proposal queing, quorum and vote accounting logic, potentially resulting in footguns for the implementers.

Overall, the codebase exhibits strong modular architecture, separation of concerns, and engineering decisions that support maintainability and integrator flexibility. However, the security profile still depends on careful integrator configuration and strict validation in critical execution paths.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.